TRACKSHIELD: AN INTELLIGENT AI-IoT SYSTEM FOR RAILWAY TRACK HEALTH AND INTRUSION DETECTION

Swetha Sri .A
Department of B.Tech CSE (AI and ML).
VISTAS, Chennai, India.
Swethasri9423@gmail.com

Kaviya Sakthivel
Department of B.Tech CSE (AI and ML).
VISTAS, Chennai, India.
kavyasakthi0506@gmail.com

Dorcas Gazelle Eunice G
Department of B.Tech (AI and ML)
VISTAS, Chennai, India
gazelle9712@gmail.com

Abstract— Railway safety depends on timely observation of track-related abnormalities and unauthorized access to restricted areas. This paper presents TrackShield, a student-scale AI-IoT prototype concept that combines physical sensing and visual intrusion monitoring. An ESP32 acts as the central controller; an ADXL345 accelerometer measures movement or tilt, an SW-420 module indicates vibration events, and a HuskyLens AI camera provides configured visual recognition. When an event rule is satisfied, the controller can activate a buzzer, LED and LCD, and transmit selected event information through built-in Wi-Fi to an IoT dashboard. The paper describes the architecture, methodology, prototype setup, a proposed validation matrix and illustrative event-log rows. Since measured trial results were not supplied, illustrative values are not presented as experimental findings. TrackShield is intended for academic demonstration and is not a certified railway safety system.

Keywords— Railway safety, track monitoring, intrusion detection, ESP32, ADXL345, SW-420, HuskyLens, Internet of Things.

I. INTRODUCTION

Railway networks are critical transportation infrastructure, and safe operation requires observation of both track condition and the surrounding restricted area. Abnormal vibration or movement may indicate a condition requiring inspection, while a person or object entering a track zone may create a hazard. Separate monitoring methods can make it difficult to combine physical condition information and visual awareness in one compact system.

TrackShield combines sensor-based track monitoring, AI-assisted visual detection, local alerts and optional remote reporting. The ESP32 serves as the controller and provides built-in Wi-Fi. The ADXL345 and SW-420 supply complementary physical inputs, while HuskyLens provides a configured visual detection function. The prototype is intended for a model railway track in a controlled academic setting, not installation on an operational railway.

The project contribution is an integrated low-cost architecture in which heterogeneous inputs share a common warning workflow. A repeatable test matrix and event-recording format are also proposed so that actual observations can be collected and reported transparently.

II. PROBLEM STATEMENT AND OBJECTIVES

A vibration sensor alone cannot identify who or what entered a restricted zone, while a camera does not directly measure movement of the track structure. TrackShield combines both modalities under one controller to provide complementary monitoring. The objectives are:

- Monitor movement or tilt using an ADXL345 accelerometer mounted on the model track or support.
- Detect vibration events using an SW-420 vibration module.
- Use HuskyLens to detect or recognize configured people or objects in a restricted demonstration area.
- Process sensor and camera states through an ESP32 and generate buzzer, LED and LCD alerts.
- Transmit selected status and event information to an IoT dashboard when Wi-Fi and software integration are available.
- Define a test plan without making unsupported claims about operational railway performance.

III. LITERATURE SURVEY

Prior work motivates the use of IoT sensing and machine vision for railway safety. Chellaswamy et al. described an IoT-based rail-track condition monitoring and derailment-prevention system using accelerometer and location information [1]. PatilPillai et al. investigated IoT-based condition monitoring for railway track fault detection [2]. These works motivate sensor-driven monitoring, while the present concept adds a separate visual intrusion input.

Kumar and Harsha reviewed machine-vision-based railway defect detection and discussed evaluation challenges [3]. Cao et al. presented a railway intrusion detection method using a dynamic intrusion region and lightweight neural network [4]. These studies support visual monitoring but also underline the importance of datasets, benchmark conditions and deployment validation.

Ref.	Research direction	Relevance to TrackShield	Consideration
[1]	IoT track monitoring with accelerometer/location data	Supports sensor-driven monitoring.	Model-track tests cannot establish field reliability.
[2]	Inertial sensing for track fault monitoring	Motivates movement and vibration sensing.	Mounting and test conditions affect readings.
[3]	Review of machine-vision railway inspection	Highlights vision and evaluation needs.	Datasets and real-time benchmarks matter.
[4]	Railway intrusion detection using lightweight neural network	Motivates restricted-zone detection.	Lighting, viewpoint and target setup affect results.

Table I. Selected literature relevant to the TrackShield concept.

IV. METHODOLOGY

A. System architecture

The architecture contains three sensing inputs, a central controller, local indicators and an optional IoT reporting layer. The ESP32-side application interprets sensor and camera states, applies configured thresholds or event rules, produces a local warning and records the event. The camera recognition mode and target classes must be configured and documented during implementation.

SENSING INPUTS	CONTROLLER / DECISION	OUTPUTS
ADXL345: movement/tilt SW-420: vibration event HuskyLens: configured visual detection	ESP32 Read inputs Evaluate event rules Log event state	Local: buzzer, LED, LCD Built-in Wi-Fi IoT dashboard / event log

Fig. 1. Functional architecture of the proposed TrackShield academic prototype.

The data path is: (1) acquire movement/vibration signals and camera state; (2) validate or smooth inputs if implemented; (3) evaluate event rules; (4) update the LCD and activate the buzzer/LED; and (5) transmit status to the IoT platform if connected. Local warning should not depend exclusively on cloud connectivity.

B. Hardware and software requirements

Component	Role	Integration note
ESP32 development board	Main controller and Wi-Fi communication.	Check GPIO compatibility and logic levels.
ADXL345 accelerometer	Measures acceleration and movement/tilt.	Typically uses I²C or SPI; document orientation.
SW-420 vibration module	Indicates vibration events.	Sensitivity depends on adjustment and mounting.
HuskyLens AI camera	Configured visual detection/recognition.	Fix camera and define recognition mode.
16×2 LCD, buzzer, LED	Local status and warning.	Use suitable drivers and current limiting.
IoT dashboard	Remote status and event monitoring.	Requires network and firmware integration.

Table II. Hardware roles and implementation considerations.

C. Sensor acquisition and visual monitoring

The ADXL345 should be attached firmly to the model track or support. Acceleration values are sampled on configured axes; changes relative to a baseline may indicate movement or tilt. Thresholds must be established experimentally for the assembled model. The SW-420 provides a vibration event signal, but nearby movement, mounting and sensitivity adjustment can affect its response.

HuskyLens should be mounted beside the model track and directed toward the restricted zone. Its recognition mode is configured for the intended demonstration target. A detection is treated as an intrusion-related event only when it meets the project's configured rule. Performance can vary with lighting, distance, occlusion and viewpoint.

D. Event decisions and alerts

The ESP32 combines sensor states and camera events using a rule-based process. A vibration event can trigger a track-monitoring warning, while a configured camera detection can trigger an intrusion warning. If both occur close together, the display can report a combined alert. Debouncing or a short persistence check may reduce repeated warnings from transient noise. The final implementation should document the rules actually used in firmware.

V. PROTOTYPE SETUP AND WORKFLOW

For an academic demonstration, place the sensors on a low-voltage model-track assembly and fix the camera so that the test zone remains in view. Connect the sensor outputs and local indicators to the ESP32 according to the selected module interfaces. Implement separate firmware functions for data acquisition, event decisions, display/alert handling and Wi-Fi reporting. Record timestamp, acceleration, vibration state, camera state, alert category and network-delivery status for every trial.

A practical workflow is: initialise devices; capture baseline readings; read sensor and camera states; apply event rules; activate local warnings; send event status if network connectivity is available; and append the observation to the log. Test local alert behavior separately from dashboard connectivity so that a network outage does not conceal controller behavior.

VI. DATASET AND ILLUSTRATIVE EVENT LOG

No measured dataset was supplied with the project material. The rows below are therefore illustrative examples showing how a future event log may be structured; they are not experimental results, a labelled training dataset or evidence of achieved detection accuracy. Actual values must be captured from the assembled prototype.

ID	Acceleration example	SW-420	Camera state	Expected event label
S1	0.98 g	0	0	Normal
S2	1.18 g	1	0	Vibration warning
S3	1.02 g	0	1	Intrusion warning
S4	1.21 g	1	1	Combined warning

Table III. Illustrative event-log examples only; values are not measured results.

For each real test, record sensor readings, expected state, observed state, alert activation, IoT delivery result, false alarms and missed detections. Repeat each condition and vary camera distance, viewing angle, lighting, mounting and vibration intensity.

VII. TESTING AND PERFORMANCE EVALUATION

Test	Condition	Expected output	Evidence to record
T1	Stable track; no target	Normal; no warning	Baseline readings and sensor states.
T2	Apply controlled vibration	Vibration warning	Trigger count, response time, false triggers.
T3	Place configured target in view	Intrusion warning	Detection success, distance and lighting.
T4	Vibration and target together	Combined warning	Combined label and alert response.
T5	Disconnect Wi-Fi	Local alerts continue	Local behavior and remote-delivery status.
T6	Repeat a condition	Consistent response	Trial count and variation.

Table IV. Proposed validation test matrix.

For a defined event class, useful metrics include detection rate = TP/(TP+FN) × 100%; false-alarm rate = FP/(FP+TN) × 100%; missed-event rate = FN/(TP+FN) × 100%; mean alert latency = sum of alert delays / number of detected trials; and IoT delivery success = messages received / messages sent × 100%. TP denotes true positives, FN false negatives, FP false positives and TN true negatives. Calculate these values only after recording actual trial outcomes.

Metric	Calculation / definition	Purpose
Detection rate	TP / (TP + FN)	How often target events are detected.
False-alarm rate	FP / (FP + TN)	Measures unnecessary warnings.
Missed-event rate	FN / (TP + FN)	Measures events that did not trigger.
Alert latency	Time from event to local alert	Measures responsiveness.
IoT delivery success	Messages received / messages sent	Measures dashboard communication.

Table V. Recommended evaluation metrics.

The current project description does not establish measured classifier accuracy or field performance. If a machine-learning classifier is added later, document the dataset labels, train/test split, model settings and per-class precision and recall.

VIII. EXPECTED RESULTS AND DISCUSSION

The intended demonstration is an integrated response to separate physical and visual events. Under a stable baseline, the system should show normal status. A vibration event should produce a track-monitoring warning, while a configured visual target should produce an intrusion warning. If both occur, the controller should report a combined event. During network loss, local indicators should continue operating while remote status indicates disconnection.

These are expected behaviors derived from the proposed design, not verified experimental findings. Actual performance will depend on sensor mounting, threshold adjustment, camera configuration, firmware and Wi-Fi conditions. The results section should be completed after executing the test matrix and should include successful and unsuccessful trials.

IX. ADVANTAGES AND LIMITATIONS

Advantages: TrackShield combines physical track sensing and camera-based restricted-area monitoring; uses ESP32 built-in Wi-Fi for optional remote status reporting; provides local buzzer, LED and LCD feedback; and uses modular hardware that can be changed as the project develops.

Limitations: This is an academic prototype and is not validated for operational railway use. The SW-420 indicates vibration but does not tify its cause. Accelerometer readings depend on mounting and ironmental movement. Camera performance can vary with lighting, ance, viewpoint and occlusion. Remote monitoring depends on ork availability and correct dashboard integration. No measured field set, detection accuracy or latency results were supplied.

X. FUTURE SCOPE

re work can include a labelled dataset from repeated model-track experiments, signal filtering and adaptive thresholds, a more robust intrusion-detection model, event localization, timestamped cloud logging and a dashboard for historical review. Evaluation should cover vibration patterns, target types, camera angles and environmental conditions. If machine learning is added, use separated training and test data and compare with a transparent rule-based baseline. Any transition toward railway use would require domain engineering, fail-safe design, field validation and applicable certification.

XI. CONCLUSION

ckShield presents an AI-IoT railway safety monitoring concept that mbines ADXL345 movement sensing, SW-420 vibration events and kyLens visual detection under an ESP32 controller. Local buzzer, and LCD indicators provide immediate feedback, while built-in Wi-Fi support remote status reporting. This paper follows a ference-style structure with a literature survey, methodology, trative event-log format, proposed test matrix and evaluation metrics. next step is to execute the tests on the physical model, replace strative values with observed readings, and report detection rate, false ms and alert latency. TrackShield should remain an academic otype until more rigorous validation is completed.

REFERENCES

[1] C. Chellaswamy, T. S. Geetha, A. Vanathi, and K. Venkatachalam, “An IoT based rail track condition monitoring and derailment prevention system,” *International Journal of RF Technologies: Research and Applications*, vol. 11, no. 2, pp. 81–107, 2020, doi: 10.3233/RFT-190210.

[2] T. PatilPillai et al., “IoT Based Condition Monitoring for Railway Track Fault Detection in Smart Cities,” *IETE Journal of Research*, vol. 69, no. 9, pp. 5794–5803, 2023, doi: 10.1080/03772063.2022.2150701.

[3] A. Kumar and S. P. Harsha, “A systematic literature review of defect detection in railways using machine vision-based inspection methods,” *International Journal of Transportation Science and Technology*, vol. 18, pp. 207–226, 2025, doi: 10.1016/j.ijtst.2024.06.006.

[4] Z. Cao, Y. Qin, Z. Xie, Q. Liu, E. Zhang, Z. Wu, and Z. Yu, “An effective railway intrusion detection method using dynamic intrusion region and lightweight neural network,” *Measurement*, vol. 191, Art. no. 110564, 2022, doi: 10.1016/j.measurement.2021.110564.

Preparation note: This draft adapts the structure of the supplied reference conference paper while retaining TrackShield’s topic and references. Verify all bibliographic details and author information. Replace illustrative sample rows with actual prototype observations before submission.